

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ:~/skill$ nano skill4.c
saketh@DESKTOP-OQ0P5LQ:~/skill$ gcc skill4.c -o skill4
saketh@DESKTOP-OQ0P5LQ:~/skill$ ./skill4
Simple Shell Tokenizer and Parser
Enter command: ls
```

```
--- Tokens ---
Token 1: WORD      Value: ls
Token 2: END       Value: END
Syntax is valid.
```

```
--- Parse Tree ---
COMMAND
  ls
```

```
--- Execution Structure ---
Commands: [ls]
```

```
Parsing completed successfully.
```

```
saketh@DESKTOP-OQ0P5LQ:  ×  +  v
GNU nano 7.2 skill4.c *
/* Remove newline */
input[strcspn(input, "\n")] = '\0';

/* Handle empty command */
if (strlen(input) == 0) {
    printf("Empty command.\n");
    return 0;
}

/* Tokenize */
tokenize(input, tokens, &count);

/* Display tokens */
print_tokens(tokens, count);

/* Validate syntax */
if (!validate_syntax(tokens, count))
    return 1;

/* Build parse tree */
ParseNode *tree = build_parse_tree(tokens, count);

printf("\n--- Parse Tree ---\n");

print_tree(tree, 0);

/* Execution structure */
create_execution_structure(tokens, count);

/* Release memory */
free_tree(tree);

printf("\nParsing completed successfully.\n");

return 0;
}
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: x + v
GNU nano 7.2 skill4.c *
    printf(" INPUT ");

    else if (tokens[i].type == TOKEN_OUTPUT)
        printf(" OUTPUT ");

    else if (tokens[i].type == TOKEN_APPEND)
        printf(" APPEND ");
    }
    printf("\n");
}

/* Main */
int main() {

    char input[500];

    Token tokens[MAX_TOKENS];

    int count = 0;

    printf("Simple Shell Tokenizer and Parser\n");

    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    /* Tokenize */
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: × + ∨
GNU nano 7.2 skill4.c *

    return root;
}

/* Print tree */
void print_tree(ParseNode *root, int level) {

    if (root == NULL)
        return;

    for (int i = 0; i < level; i++)
        printf(" ");

    printf("%s\n", root->value);

    print_tree(root->left, level + 1);
    print_tree(root->right, level + 1);
}

/* Execution structure */
void create_execution_structure(Token tokens[], int count) {

    printf("\n--- Execution Structure ---\n");

    printf("Commands: ");

    for (int i = 0; i < count - 1; i++) {

        if (tokens[i].type == TOKEN_WORD)
            printf("[%s] ", tokens[i].value);

        else if (tokens[i].type == TOKEN_PIPE)
            printf(" -> PIPE -> ");

        else if (tokens[i].type == TOKEN_INPUT)
            printf(" INPUT ");
    }
}
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: × + ▾
GNU nano 7.2 skill4.c *

        printf("Syntax Error: Pipe requires a command after it.\n");
        return 0;
    }
}

/* Command cannot end with pipe */
if (tokens[count - 2].type == TOKEN_PIPE) {
    printf("Syntax Error: Command cannot end with '|'.\n");
    return 0;
}

printf("Syntax is valid.\n");

return 1;
}

/* Build simple parse tree */
ParseNode *build_parse_tree(Token tokens[], int count) {

    ParseNode *root = create_node("COMMAND");

    ParseNode *current = root;

    for (int i = 0; i < count - 1; i++) {

        if (tokens[i].type == TOKEN_WORD) {

            ParseNode *node = create_node(tokens[i].value);

            if (current->left == NULL)
                current->left = node;
            else
                current->right = node;
        }

        else if (tokens[i].type == TOKEN_PIPE) {
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: × + v
GNU nano 7.2 skill4.c *
int validate_syntax(Token tokens[], int count) {

    if (count <= 1) {
        printf("Empty command.\n");
        return 0;
    }

    /* Command cannot start with pipe */
    if (tokens[0].type == TOKEN_PIPE) {
        printf("Syntax Error: Command cannot start with '|'.\n");
        return 0;
    }

    for (int i = 0; i < count - 1; i++) {

        /* Two pipes together */
        if (tokens[i].type == TOKEN_PIPE &&
            tokens[i + 1].type == TOKEN_PIPE) {

            printf("Syntax Error: Consecutive pipes.\n");
            return 0;
        }

        /* Redirection must have a filename */
        if ((tokens[i].type == TOKEN_INPUT ||
            tokens[i].type == TOKEN_OUTPUT ||
            tokens[i].type == TOKEN_APPEND) &&
            tokens[i + 1].type != TOKEN_WORD) {

            printf("Syntax Error: Redirection requires a filename.\n");
            return 0;
        }

        /* Pipe must have command after it */
        if (tokens[i].type == TOKEN_PIPE &&
            tokens[i + 1].type != TOKEN_WORD) {
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: × + v
GNU nano 7.2 skill4.c *
        i + 1,
        token_name(tokens[i].type),
        tokens[i].value);
    }
}

/* Create parser node */
ParseNode *create_node(const char *value) {
    ParseNode *node = malloc(sizeof(ParseNode));

    if (node == NULL) {
        perror("malloc");
        exit(1);
    }

    strcpy(node->value, value);

    node->left = NULL;
    node->right = NULL;

    return node;
}

/* Free parse tree */
void free_tree(ParseNode *root) {
    if (root == NULL)
        return;

    free_tree(root->left);
    free_tree(root->right);

    free(root);
}

/* Simple syntax validation */
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: x + v
GNU nano 7.2 skill4.c *
}

    continue;
}

/* Normal word */
char word[100];
int j = 0;

while (input[i] != '\0' &&
       !isspace(input[i]) &&
       input[i] != '|' &&
       input[i] != '<' &&
       input[i] != '>') {

    if (j < 99)
        word[j++] = input[i];

    i++;
}

word[j] = '\0';

if (j > 0)
    add_token(tokens, count, TOKEN_WORD, word);
}

add_token(tokens, count, TOKEN_END, "END");
}

/* Display tokens */
void print_tokens(Token tokens[], int count) {

    printf("\n--- Tokens ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %-10s Value: %s\n",
|

saketh@DESKTOP-OQ0P5LQ: x + v
GNU nano 7.2 skill4.c *

/* Tokenizer */
void tokenize(char *input, Token tokens[], int *count) {

    int i = 0;

    while (input[i] != '\0') {

        /* Ignore whitespace */
        if (isspace(input[i])) {
            i++;
            continue;
        }

        /* Pipe */
        if (input[i] == '|') {
            add_token(tokens, count, TOKEN_PIPE, "|");
            i++;
            continue;
        }

        /* Input redirection */
        if (input[i] == '<') {
            add_token(tokens, count, TOKEN_INPUT, "<");
            i++;
            continue;
        }

        /* Output redirection */
        if (input[i] == '>') {

            if (input[i + 1] == '>') {
                add_token(tokens, count, TOKEN_APPEND, ">>");
                i += 2;
            }
            else {
                add_token(tokens, count, TOKEN_OUTPUT, ">");
|
```

SKILL-4

```
saketh@DESKTOP-OQ0P5LQ: × + v
GNU nano 7.2 skill4.c *

    case TOKEN_PIPE:
        return "PIPE";

    case TOKEN_INPUT:
        return "INPUT";

    case TOKEN_OUTPUT:
        return "OUTPUT";

    case TOKEN_APPEND:
        return "APPEND";

    case TOKEN_END:
        return "END";

    default:
        return "UNKNOWN";
}
}

/* Add token */
void add_token(Token tokens[], int *count,
               TokenType type, const char *value) {

    if (*count >= MAX_TOKENS - 1) {
        printf("Too many tokens!\n");
        return;
    }

    tokens[*count].type = type;

    strncpy(tokens[*count].value, value, 99);
    tokens[*count].value[99] = '\0';

    (*count)++;
}
}
```

```
saketh@DESKTOP-OQ0P5LQ: × + v
GNU nano 7.2 skill4.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100

/* Token types */
typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_INPUT,
    TOKEN_OUTPUT,
    TOKEN_APPEND,
    TOKEN_END
} TokenType;

/* Token structure */
typedef struct {
    TokenType type;
    char value[100];
} Token;

/* Parser node */
typedef struct ParseNode {
    char value[100];
    struct ParseNode *left;
    struct ParseNode *right;
} ParseNode;

/* Convert token type to string */
const char *token_name(TokenType type) {

    switch (type) {

        case TOKEN_WORD:
            return "WORD";
    }
}
```